Custom Field Implementation (Frontend & Backend)

This document describes the complete and standardized process for implementing **Custom Fields** across frontend and backend, including prerequisites, database setup, frontend integration, backend handling, and list-view rendering.

---

1. Affected Database Tables

The following tables are involved in the Custom Field module:

- `root_cf_input_types`
- `root_cf_api_inputs`
- `root_cf_forms`
- `root_cf_form_positions`
- `root_cf_form_trees`
- `root_cf_form_inputs`
- `root_cf_form_input_options`
- `root_cf_form_input_mapping`
- `root_cf_form_values`

---

2. Pre-requisites

Before implementing Custom Fields, ensure the following steps are completed:

2.1 Seed the Custom Field Module

```
php artisan app-module:seed AppSmartOffice CustomFieldModule
```

2.2 Register the Form

Insert a record into `root_cf_forms` for the form where custom fields will be applied.

- `table_name` → Reference table name
- `code` → Unique identifier for the form

2.3 Define Form Position

Insert a record into `root_cf_form_positions`.

- Usual placement: **bottom** of the form

2.4 Register Form in Tree Structure

Insert the form into `root_cf_form_trees` to generate an application-wise tree structure.

This allows the form to appear in **Settings → Custom Fields**.

---

3. Step 1: Creating Custom Fields in Create / Edit Forms

3.1 Frontend Implementation

3.1.1 State Initialization

```
const [isCustomFieldAvailable, setIsCustomFieldAvailable] = useState(false);
const [customFieldValidate, setCustomFieldValidate] = useState(() => () =>
({}));
const [customFieldData, setCustomFieldData] = useState(false);
const { config } = useAppConfig();
```

3.1.2 Fetch Custom Field Configuration

```
const fetchResponseInputType = useCallback(async () => {
    const formInformation = await fetchData(
        CUSTOM_FIELD_ENDPOINTS.IS_CUSTOM_FIELDS,
        {
            code: "sample_code", // root_cf_forms.code
            table_row_id: "", // pass record ID for edit page
        }
    );

    if (formInformation?.data) {
        setIsCustomFieldAvailable(true);
        setCustomFieldData(formInformation.data);
    }
}, [config?.isCustomFieldSubmit]);

useEffect(() => {
```

```
    fetchResponseInputType();
}, [fetchResponseInputType]);
```

3.1.3 Attach Custom Field Validation to Formik

Add the following prop to the Formik component:

```
validate={customFieldValidate}
```

3.1.4 Render Custom Field Inputs and Drawer

```
<CustomFieldInputsAndDrawer
    key={"bottom"}
    isCustomFieldAvailable={isCustomFieldAvailable}
    isCustomFieldInputs={true}
    isCustomFieldDrawer={true}
    values={values}
    isAddExisting={true}
    customFieldData={customFieldData}
    position={"bottom"}
    setInitialValues={setInitialValues}
    setCustomFieldValidate={setCustomFieldValidate}
/>
```

This component: - Displays custom fields - Handles creation and validation - Manages drawer-based UI

3.2 Form Submission Handling

3.2.1 Pass Custom Field Values

Include custom field data in the create/update API request:

```
values?.custom_fields
```

3.2.2 Store Custom Field Values (Backend)

After creating or updating the main record, call the Custom Field service:

```
app(CustomFieldService::class)
    ->storeCustomFieldInputValue($request, $data['id']);
```

- `$data['id']` → Reference table row ID

3.3 Request Header Requirement

Both **Create** and **Update** APIs must include:

```
Content-Type: multipart/form-data
```

---

4. Step 2: Display Custom Fields in Frontend List View

4.1 Frontend Data Table Configuration

Pass `doc_type` to the `DataTableComponent`.

- `doc_type` = `root_cf_forms.table_name`

This allows the frontend to fetch and display custom field columns dynamically.

4.2 Backend Model Configuration

4.2.1 Extend BaseModel

Ensure the model extends `BaseModel`:

```
class ExampleModel extends BaseModel
{
    // ...
}
```

4.2.2 Use `selectedAttribute()` Instead of `select()`

```
Model::selectedAttribute($fields)
```

This ensures: - Custom field attributes are included - Dynamic field mapping works correctly

5. Summary

- Custom Fields require both **DB registration** and **frontend integration**
- Forms must be registered in `root_cf_forms`, `root_cf_form_positions`, and `root_cf_form_trees`
- Frontend must dynamically fetch and render custom fields
- Backend must store and retrieve custom field values using `CustomFieldService`
- List views require `doc_type` and `BaseModel` integration

This structure ensures a scalable, reusable, and consistent Custom Field implementation across the application.